

Übersetzung und Analyse von Programmen

A stylized, low-poly illustration of a university building with large windows and a red abstract sculpture. In the foreground, there are two green trees and a large yellow flower with a blue center. The sky is blue with a large yellow sun and green geometric shapes.

Vorlesung im Wintersemester 2025/26

Prof. Dr. Stephan Diehl
Informatik, Universität Trier

Übersetzung von TRIPLA

Projekt

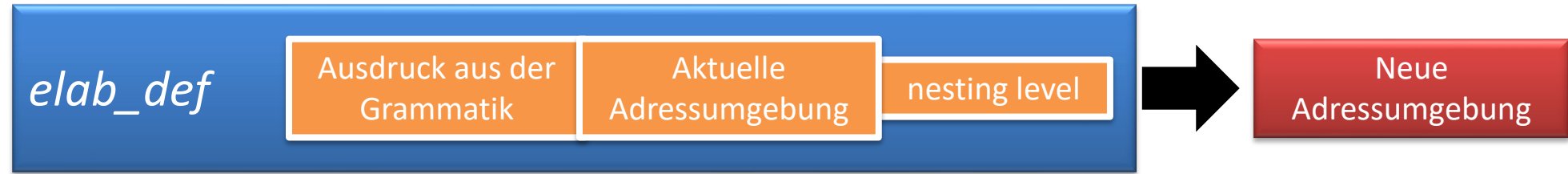
TRAM implementieren

Lexikalische Analyse implementieren
Parser implementieren

Berechnung der Adressumgebung
Erzeugung von TRAM-Code

PROJEKT III – CODEERZEUGUNG: TRIPLA → TRAM

Berechnung der Adressumgebung



- $elab_def (d_1 \dots d_k) \rho \, nl = \rho_k$ **where** $\rho_0 = \rho$ and $\rho_i = elab_def \, d_i \, \rho_{i-1} \, nl$



- $elab_def (id(id_1, \dots, id_k)\{e\}) \rho \, nl = \rho[(\ell, nl)/id]$
where ℓ is a new label

Eintragen von *id* mit dem Wert (ℓ, nl) in die Adressumgebung.
Falls *id* bereits enthalten ist, wird der existierende Wert überschrieben.

Codeerzeugung - Labels



Übersetzungsschemata, die ein Label auf den Code eines Kindknotens setzen, bedürfen weiterer Überlegungen.
Beispiel Funktionsdefinition:

'code *e*' könnte bereits ein Label an erster Instruktion gesetzt haben

code (*id*(*id*₁, ..., *id*_{*k*}){*e*}) *p* *nl* =

ℓ: *code* *e* *p*[(0,*nl*+1)/*id*₁] ... *p*[(*k*-1,*nl*+1)/*id*_{*k*}] *nl*+1;

return;

where *p*(*id*) = (*ℓ*,*nl*)

Drei Alternativen bieten sich an, um diesem Umstand gerecht zu werden:

1. **Eine Instruktion kann mit mehreren Labels versehen werden**
2. Man prüft, ob die erste Instruktion bereits ein Label besitzt und
 - a) übernimmt es oder
 - b) ersetzt es und biegt alle Instruktionen des Kindknotens um, die auf das ursprüngliche Label zeigen
3. Erweitert das Übersetzungsschema mithilfe einer NOP Instruktion



→
$$\begin{aligned} & \text{code}(\text{id}(\text{id}_1, \dots, \text{id}_k)\{e\}) \rho \text{ nl} = \\ & \quad \boxed{\ell: \text{nop}} \\ & \quad \text{code } e \rho[(0, \text{nl}+1)/\text{id}_1] \dots \rho[(k-1, \text{nl}+1)/\text{id}_k] \text{ nl}+1; \\ & \quad \text{return} \\ & \quad \text{where } \rho(\text{id}) = (\ell, \text{nl}) \end{aligned}$$

- Das Label-Problem betrifft die Übersetzungsschemata von:
 - Fallunterscheidung: *if* c *then* e_1 *else* e_2
 - Funktionsdefinition: *id* ($id_1, \dots, id_k$) { e }
 - Let-Ausdruck: *let* d *in* e
 - Schleifen: *while* c *do* e

Codeerzeugung – Operatoren AND / OR

Die binären, logischen Operatoren AND (&&) sowie OR (||) können auf zwei unterschiedliche Weisen übersetzt werden:

- **Strikt**: Die beiden Ausdrücke werden übersetzt und danach der Operator angewandt
- **Nicht strikt**: Es wird erst ein Ausdruck übersetzt und dann entschieden, ob es noch notwendig ist den zweiten auch zu übersetzen

Je nach Implementierung, Programmlogik und Verwendung von Seiteneffekt behafteten Methoden, entstehen nicht direkt offensichtliche Defekte, die nur schwer aufzudecken sind! → Java, Python = nicht strikt, d.h.

```
if(2 < 0 && sideEffect()) ...
```

→ Möglicherweise notwendiger Seiteneffekt wird nicht ausgeführt

In Python: `if (2 < 0 and sideEffect()): ...`

Codeerzeugung - AND

Strikte Umsetzung:

```
code  $c_1$  &&  $c_2$  p nl =  
  code  $c_1$  p nl;  
  code  $c_2$  p nl;  
  mul;
```



c_i hinterlassen
entweder 0 oder 1 auf
dem Keller

Nicht strikte Umsetzung



```
code  $c_1$  &&  $c_2$  p nl =  
  code  $c_1$  p nl;  
  ifzero  $l_1$ ;  
  const 1;  
  code  $c_2$  p nl;  
  mul;  
  goto  $l_2$ ;  
 $l_1$ : const 0;  
 $l_2$ : nop;
```

where l_1, l_2 are new labels

Strikte Umsetzung:

```
code  $c_1$  ||  $c_2$  p nl =  
  code  $c_1$  p nl;  
  code  $c_2$  p nl;  
  add;  
  ifzero  $l_1$ ;  
  const 1;  
  goto  $l_2$ ;  
 $l_1$ : const 0;  
 $l_2$ : nop;
```

c_i hinterlassen
entweder 0 oder 1 auf
dem Keller

where l_1, l_2 are new labels

Nicht strikte Umsetzung



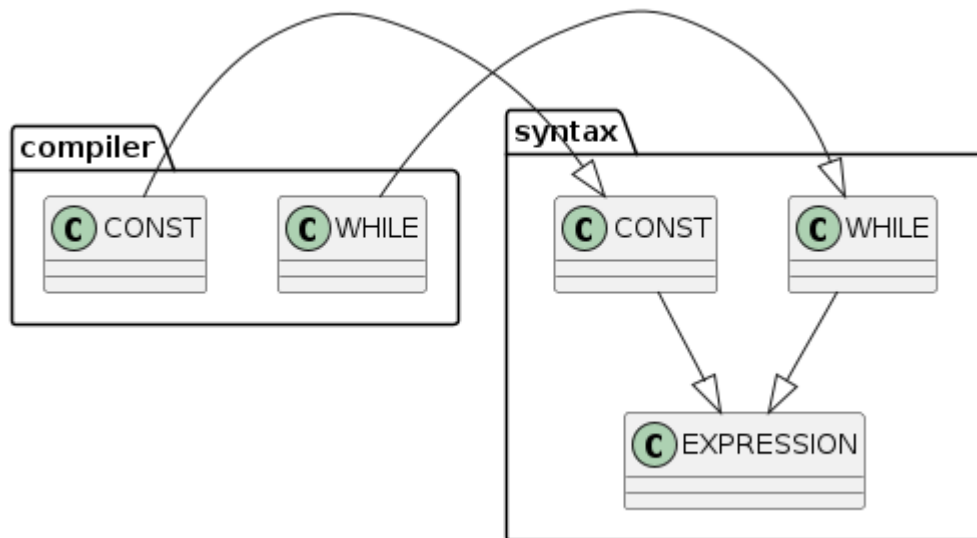
```
code  $c_1$  ||  $c_2$  p nl =  
  code  $c_1$  p nl;  
  ifzero  $l_1$ ;  
  const 1;  
  goto  $l_2$ ;  
 $l_1$ : code  $c_2$  p nl;  
 $l_2$ : nop;
```

where l_1, l_2 are new labels

Implementierung in Python

- In **triplayacc.py** müssen die Baumknoten aus **compiler.py** verwendet werden:

```
#import syntax as ast
import compiler as ast
```



- In **compiler.py** erben die Baumknoten von den gleichnamigen Baumknoten in **syntax.py** und fügen die **code-Funktion** hinzu:

```
import syntax
from instruction import *

class CONST(syntax.CONST):
    def code(self, rho, nl):
        return [const(self.value)]
```

Die Instruktionen
der TRAM sind in
instruction.py
definiert.

Visual TRAM

- python vistram.py
- Zum Testen können Sie ihren erzeugten TRAM-Code in eine Datei schreiben und dann mit der Visual TRAM einlesen und ausführen.

The screenshot shows the Visual TRAM application window. It has a title bar with a feather icon and the text 'Visual TRAM'. Below the title bar are five buttons: 'Start', 'Stop', 'Step' (which is highlighted with a dashed border), 'Reset', and 'Open File'. The main area is divided into four panels. The left panel shows a list of instructions: 00: CONST 10, 01: INVOKE 1 LSQUARE 0, 02: HALT, -> 03: LSQUARE: LOAD 0 0 (highlighted in green), 04: LOAD 0 0, 05: MUL, 06: RETURN, 07: HALT. The second panel from the left shows register values: PC : 1, FP : 0, PP : 0, TOP : 0, and a highlighted instruction '00: 10 <-TOP <-PP <-FP'. The third panel shows register values: PC : 3, FP : 1, PP : 0, TOP : 5, and a highlighted instruction '00: 10 <-PP', '01: 2 <-FP', and '05: 0 <-TOP <-end of frame'. The right panel shows the execution state: 'Ausgeführt: INVOKE 1 LSQUARE 0' and a block of Python code: tmp_top=top, push(pc+1), push(pp), push(fp), push(spp(self.d,pp,fp)), push(sfp(self.d,pp,fp)), pp=tmp_top-self.n+1, fp=tmp_top+1, pc=self.label.address.

```
Visual TRAM
```

Start Stop Step Reset Open File

```
00:  CONST 10
01:  INVOKE 1 LSQUARE 0
02:  HALT
-> 03:  LSQUARE: LOAD 0 0
04:  LOAD 0 0
05:  MUL
06:  RETURN
07:  HALT
```

```
PC : 1
FP : 0
PP : 0
TOP: 0
00: 10 <-TOP <-PP <-FP
```

```
PC : 3
FP : 1
PP : 0
TOP: 5
00: 10 <-PP
01: 2 <-FP
02: 0
03: 0
04: 0
05: 0 <-TOP <-end of frame
```

```
Ausgeführt:  INVOKE 1 LSQUARE 0

tmp_top=top
push(pc+1)
push(pp)
push(fp)
push(spp(self.d,pp,fp))
push(sfp(self.d,pp,fp))
pp=tmp_top-self.n+1
fp=tmp_top+1
pc=self.label.address
```